

1. Project Title

RAG-Based Legacy Code Understanding and Documentation Assistant for Software Developers

2. Problem Statement

Software developers frequently work on code written by other developers. Before adding new features or fixing bugs, they must understand the existing codebase.

For small and medium-sized software projects, documentation is often incomplete, outdated, or missing entirely. New developers spend significant time reading source files, tracing function calls, and understanding dependencies.

Current code search tools can find files or keywords but cannot explain:

- How a feature is implemented
- Which files are involved
- Dependencies between modules
- Impact of modifying a particular component

The proposed system uses Retrieval-Augmented Generation (RAG) to retrieve relevant code sections and generate developer-friendly explanations grounded in the actual source code.

3. Why This Project Matters

This project addresses a real industry problem.

Examples:

- New employee onboarding
- Maintenance of legacy software
- Understanding open-source repositories
- Faster debugging
- Faster feature enhancement

Many companies spend substantial effort on knowledge transfer and documentation. This project reduces that effort.

4. Scope Definition (Important)

Included

Python projects only.

Examples:

- Flask application
- Django application
- Student Management System
- E-commerce application
- Hospital Management System

Repository size:

- 50–100 source files
- 5,000–20,000 lines of code

Not Included

To keep the project realistic:

- Multi-language repositories
- Automatic bug fixing
- Code generation
- IDE plugins
- Large enterprise codebases

5. Objectives

Primary Objectives

1. Parse source code repositories.
2. Create semantic embeddings of code.
3. Store code knowledge in vector database.
4. Retrieve relevant code chunks.
5. Generate explanations using LLM.
6. Provide file references.

Secondary Objectives

1. Dependency analysis.
2. Function call tracing.
3. Documentation generation.

6. System Architecture

1. GitHub Repository
2. Code Parser
3. Code Chunks
4. Embedding Model
5. Vector Database
6. Retriever
7. LLM
8. Developer Explanation

7. Recommended Technology Stack

Programming

Python

RAG Framework

LangChain

or

LlamaIndex

Vector Database

ChromaDB

Recommended because:

- Free
- Lightweight
- Easy setup

Embedding Models

Option 1

BAAI/bge-small-en-v1.5

Advantages:

- Open source
- Small
- Fast

Option 2

Sentence Transformers

all-MiniLM-L6-v2

LLM

Option 1

Llama 3 8B

Option 2

Mistral 7B

Run locally using:

- Ollama

Frontend

Streamlit

8. Dataset Selection

DOE-1 (Design of Experiment)

Students should compare 3 repositories.

Example:

Repository A

Student Management System

Repository B

E-commerce System

Repository C

Hospital Management System

Evaluation Criteria

Criteria	Weight
Documentation Quality	20
Number of Files	20
Complexity	20
Feature Diversity	20
Availability	20

Select repository with highest score.

9. Detailed Implementation Steps

Phase 1: Repository Understanding

Goal

Understand structure.

Student Tasks

Clone repository.

Generate:

Project Tree

Example:

```
app/  
├── routes  
├── services  
├── models  
└── database
```

Deliverable

Repository Structure Report

Phase 2: Code Parsing

Goal

Extract meaningful code blocks.

Student Tasks

Use Python AST Parser.

Extract:

- Classes
- Functions
- Methods

- Imports

Example:

```
class UserService:
```

becomes one chunk.

Store Metadata

```
{  
  "file": "user_service.py",  
  "class": "UserService",  
  "function": "authenticate"  
}
```

Deliverable

Parsed Code Dataset

Phase 3: Chunking Strategy

Goal

Create retrievable units.

DOE-2

Compare chunking methods.

Method A

Entire File

Method B

Class-Level

Method C

Function-Level

Evaluation

Test query:

How does login work?

Measure:

- Retrieval relevance
- Response quality

Expected Result

Function-Level chunking should perform best.

Phase 4: Embedding Generation

Goal

Convert code into vectors.

DOE-3

Compare embeddings.

Model A

MiniLM

Model B

BGE-Small

Evaluation Metrics

Metric	Description
Retrieval Accuracy	Relevant chunks returned
Latency	Search time
Memory Usage	RAM consumption

Deliverable

Embedding Comparison Report

Phase 5: Vector Database

Goal

Store embeddings.

DOE-4

Compare

Option A

FAISS

Option B

ChromaDB

Compare

- Speed
- Setup Complexity
- Accuracy

Expected Result

ChromaDB easier for BTech project.

Phase 6: Retrieval System

Goal

Retrieve relevant code.

Queries

How is login implemented?

How is payment processed?

Where is student registration handled?

Retrieval

Top-K Search

DOE-5

Compare:

K=3

K=5

K=10

Measure

- Precision
- Response quality

Expected Result

K=5 usually optimal.

Phase 7: LLM Integration

Goal

Generate explanations.

Prompt Template

You are a software architect.

Use only the retrieved code.

Explain:

1. Purpose
2. Inputs
3. Outputs
4. Dependencies

If information is missing say:
Not found in repository.

DOE-6

Compare Models

Model A

Llama3 8B

Model B

Mistral 7B

Evaluate

Metric
Clarity
Hallucination
Latency

Phase 8: Dependency Mapping

Goal

Generate code relationships.

Example:

LoginController

↓

AuthService

↓

UserRepository

↓

Database

Tools

NetworkX

Graphviz

Deliverable

Dependency Graph

Phase 9: Frontend Development: Streamlit Pages

Page 1

Repository Upload

Page 2

Ask Questions

Page 3

Retrieved Code

Page 4

Dependency Graph

Page 5

Auto Documentation

Phase 10: Evaluation

Test Dataset Creation

Prepare:

50 Questions

Examples:

How is authentication handled?

Which file validates payments?

How are orders stored?

Evaluation Metrics

Retrieval Precision

Relevant Chunks Retrieved

Total Chunks Retrieved

Answer Relevance

Faculty Rating

1–5 scale

Hallucination Rate

Incorrect Answers

Total Answers

Latency

Average response time.

10. Expected Outcomes

Students should demonstrate:

Functional Deliverables

Upload repository

Parse source code

Create vector database

Semantic search

AI-generated explanations

Dependency visualization

Auto documentation